```
/* ============================================================
 PACKAGES IN ORACLE PL/SQL

   Topics:
   - Package Specification
   - Package Body
   - Public Procedures
   - Public Functions
   - Package Variables
   - Package Initialization

   Labs:
   - HR Package
   - Payroll Package
   - Customer Utility Package
   ============================================================ */

SET SERVEROUTPUT ON SIZE UNLIMITED;

/* ============================================================
   1. DROP OLD OBJECTS
   ============================================================ */

BEGIN
    EXECUTE IMMEDIATE 'DROP PACKAGE hr_pkg';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP PACKAGE payroll_pkg';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP PACKAGE customer_util_pkg';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE employees';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE customers';
EXCEPTION
```

```
    WHEN OTHERS THEN NULL;
END;
/

/* ========================================================
   2. CREATE SAMPLE TABLES
   ======================================================== */

CREATE TABLE employees
(
    employee_id   NUMBER PRIMARY KEY,
    employee_name VARCHAR2(100),
    department    VARCHAR2(50),
    salary        NUMBER,
    bonus         NUMBER,
    status        VARCHAR2(20)
);

CREATE TABLE customers
(
    customer_id     NUMBER PRIMARY KEY,
    customer_name   VARCHAR2(100),
    phone           VARCHAR2(30),
    city            VARCHAR2(50),
    total_purchase  NUMBER,
    loyalty_level   VARCHAR2(20),
    status          VARCHAR2(20)
);

/* ========================================================
   3. INSERT SAMPLE DATA
   ======================================================== */

INSERT INTO employees VALUES (1, 'Aung Aung', 'IT', 900000, 0, 'ACTIVE');
INSERT INTO employees VALUES (2, 'Mg Mg', 'HR', 600000, 0, 'ACTIVE');
INSERT INTO employees VALUES (3, 'Su Su', 'Finance', 1200000, 0,
'ACTIVE');

INSERT INTO customers VALUES (1, 'ABC Trading', '091111111', 'Yangon',
5000000, NULL, 'ACTIVE');
INSERT INTO customers VALUES (2, 'Global Tech', '092222222', 'Mandalay',
2000000, NULL, 'ACTIVE');
INSERT INTO customers VALUES (3, 'Myanmar Retail', '093333333', 'Yangon',
500000, NULL, 'ACTIVE');

COMMIT;

/* ========================================================
   PACKAGE CONCEPT

   Package has 2 parts:

   1. Package Specification
      - Public procedures
```

```
      - Public functions
      - Public variables
      - Public constants

   2. Package Body
      - Implementation of procedures/functions
      - Private variables
      - Private procedures/functions
      - Initialization section
   ============================================================ */

/* ============================================================
   LAB 1: HR PACKAGE
   ============================================================ */

CREATE OR REPLACE PACKAGE hr_pkg
AS
    /* Public package variable */
    g_company_name VARCHAR2(100) := 'Realistic Infotech Group';

    /* Public procedure */
    PROCEDURE add_employee
    (
        p_employee_id   IN employees.employee_id%TYPE,
        p_employee_name IN employees.employee_name%TYPE,
        p_department    IN employees.department%TYPE,
        p_salary        IN employees.salary%TYPE
    );

    /* Public procedure */
    PROCEDURE show_employee
    (
        p_employee_id IN employees.employee_id%TYPE
    );

    /* Public function */
    FUNCTION get_employee_salary
    (
        p_employee_id IN employees.employee_id%TYPE
    )
    RETURN NUMBER;

    /* Public function */
    FUNCTION get_employee_status
    (
        p_employee_id IN employees.employee_id%TYPE
    )
    RETURN VARCHAR2;
END hr_pkg;
/

CREATE OR REPLACE PACKAGE BODY hr_pkg
AS
    /* Private package variable */
```

```
g_total_employees NUMBER := 0;

/* Private procedure */
PROCEDURE print_line
AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('---------------------------------');
END;

PROCEDURE add_employee
(
    p_employee_id   IN employees.employee_id%TYPE,
    p_employee_name IN employees.employee_name%TYPE,
    p_department    IN employees.department%TYPE,
    p_salary        IN employees.salary%TYPE
)
AS
BEGIN
    INSERT INTO employees
    (
        employee_id,
        employee_name,
        department,
        salary,
        bonus,
        status
    )
    VALUES
    (
        p_employee_id,
        p_employee_name,
        p_department,
        p_salary,
        0,
        'ACTIVE'
    );

    COMMIT;

    g_total_employees := g_total_employees + 1;

    DBMS_OUTPUT.PUT_LINE('Employee added successfully.');
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Employee ID already exists.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('HR Package Error: ' || SQLERRM);
END add_employee;

PROCEDURE show_employee
(
    p_employee_id IN employees.employee_id%TYPE
)
AS
```

```
    v_emp employees%ROWTYPE;
BEGIN
    SELECT *
    INTO v_emp
    FROM employees
    WHERE employee_id = p_employee_id;

    print_line;
    DBMS_OUTPUT.PUT_LINE('Company: ' || g_company_name);
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_emp.employee_id);
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_emp.employee_name);
    DBMS_OUTPUT.PUT_LINE('Department: ' || v_emp.department);
    DBMS_OUTPUT.PUT_LINE('Salary: ' || v_emp.salary);
    DBMS_OUTPUT.PUT_LINE('Status: ' || v_emp.status);
    print_line;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found.');
END show_employee;

FUNCTION get_employee_salary
(
    p_employee_id IN employees.employee_id%TYPE
)
RETURN NUMBER
AS
    v_salary employees.salary%TYPE;
BEGIN
    SELECT salary
    INTO v_salary
    FROM employees
    WHERE employee_id = p_employee_id;

    RETURN v_salary;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 0;
END get_employee_salary;

FUNCTION get_employee_status
(
    p_employee_id IN employees.employee_id%TYPE
)
RETURN VARCHAR2
AS
    v_status employees.status%TYPE;
BEGIN
    SELECT status
    INTO v_status
    FROM employees
    WHERE employee_id = p_employee_id;
```

```
        RETURN v_status;

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN 'NOT FOUND';
    END get_employee_status;

BEGIN
    /* Package Initialization Section */
    SELECT COUNT(*)
    INTO g_total_employees
    FROM employees;

    DBMS_OUTPUT.PUT_LINE('HR Package initialized.');
    DBMS_OUTPUT.PUT_LINE('Current Employee Count: ' ||
g_total_employees);
END hr_pkg;
/

/* Calling HR Package */
BEGIN
    hr_pkg.show_employee(1);

    hr_pkg.add_employee
    (
        4,
        'Hla Hla',
        'Sales',
        700000
    );

    hr_pkg.show_employee(4);

    DBMS_OUTPUT.PUT_LINE(
        'Employee Salary: ' || hr_pkg.get_employee_salary(4)
    );

    DBMS_OUTPUT.PUT_LINE(
        'Employee Status: ' || hr_pkg.get_employee_status(4)
    );
END;
/
```

```
HR Package initialized.
Current Employee Count: 3
-------------------------------
Company: Realistic Infotech Group
Employee ID: 1
Employee Name: Aung Aung
Department: IT
Salary: 900000
Status: ACTIVE
-------------------------------
Employee added successfully.
-------------------------------
Company: Realistic Infotech Group
Employee ID: 4
Employee Name: Hla Hla
Department: Sales
Salary: 700000
Status: ACTIVE
-------------------------------
Employee Salary: 700000
Employee Status: ACTIVE

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   LAB 2: PAYROLL PACKAGE
   ============================================================ */

CREATE OR REPLACE PACKAGE payroll_pkg
AS
    /* Public constants */
    c_tax_rate   CONSTANT NUMBER := 0.05;
    c_bonus_rate CONSTANT NUMBER := 0.10;

    PROCEDURE calculate_bonus
    (
        p_employee_id IN employees.employee_id%TYPE
    );

    PROCEDURE process_salary
    (
        p_employee_id IN employees.employee_id%TYPE
    );

    FUNCTION calculate_tax
    (
        p_salary IN NUMBER
    )
    RETURN NUMBER;

    FUNCTION calculate_net_salary
    (
        p_salary IN NUMBER
```

```
    )
    RETURN NUMBER;
END payroll_pkg;
/

CREATE OR REPLACE PACKAGE BODY payroll_pkg
AS
    FUNCTION calculate_tax
    (
        p_salary IN NUMBER
    )
    RETURN NUMBER
    AS
    BEGIN
        RETURN p_salary * c_tax_rate;
    END calculate_tax;

    FUNCTION calculate_net_salary
    (
        p_salary IN NUMBER
    )
    RETURN NUMBER
    AS
        v_tax NUMBER;
    BEGIN
        v_tax := calculate_tax(p_salary);

        RETURN p_salary - v_tax;
    END calculate_net_salary;

    PROCEDURE calculate_bonus
    (
        p_employee_id IN employees.employee_id%TYPE
    )
    AS
        v_salary employees.salary%TYPE;
        v_bonus  employees.bonus%TYPE;
    BEGIN
        SELECT salary
        INTO v_salary
        FROM employees
        WHERE employee_id = p_employee_id
        AND status = 'ACTIVE';

        v_bonus := v_salary * c_bonus_rate;

        UPDATE employees
        SET bonus = v_bonus
        WHERE employee_id = p_employee_id;

        COMMIT;

        DBMS_OUTPUT.PUT_LINE('Bonus calculated successfully.');
        DBMS_OUTPUT.PUT_LINE('Bonus Amount: ' || v_bonus);
```

```
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Active employee not found.');
    END calculate_bonus;

    PROCEDURE process_salary
    (
        p_employee_id IN employees.employee_id%TYPE
    )
    AS
        v_emp        employees%ROWTYPE;
        v_tax        NUMBER;
        v_net_salary NUMBER;
    BEGIN
        SELECT *
        INTO v_emp
        FROM employees
        WHERE employee_id = p_employee_id;

        v_tax := calculate_tax(v_emp.salary);
        v_net_salary := calculate_net_salary(v_emp.salary);

        DBMS_OUTPUT.PUT_LINE('===== Payroll Result =====');
        DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_emp.employee_name);
        DBMS_OUTPUT.PUT_LINE('Gross Salary: ' || v_emp.salary);
        DBMS_OUTPUT.PUT_LINE('Tax: ' || v_tax);
        DBMS_OUTPUT.PUT_LINE('Net Salary: ' || v_net_salary);

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Employee not found.');
    END process_salary;

BEGIN
    DBMS_OUTPUT.PUT_LINE('Payroll Package initialized.');
END payroll_pkg;
/

/* Calling Payroll Package */
BEGIN
    payroll_pkg.calculate_bonus(1);
    payroll_pkg.process_salary(1);

    DBMS_OUTPUT.PUT_LINE(
        'Tax for 1,000,000: ' || payroll_pkg.calculate_tax(1000000)
    );

    DBMS_OUTPUT.PUT_LINE(
        'Net Salary for 1,000,000: ' ||
payroll_pkg.calculate_net_salary(1000000)
    );
END;
/
```

```
Payroll Package initialized.
Bonus calculated successfully.
Bonus Amount: 90000
===== Payroll Result =====
Employee Name: Aung Aung
Gross Salary: 900000
Tax: 45000
Net Salary: 855000
Tax for 1,000,000: 50000
Net Salary for 1,000,000: 950000

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   LAB 3: CUSTOMER UTILITY PACKAGE
   ============================================================ */

CREATE OR REPLACE PACKAGE customer_util_pkg
AS
    g_default_status VARCHAR2(20) := 'ACTIVE';

    PROCEDURE add_customer
    (
        p_customer_id    IN customers.customer_id%TYPE,
        p_customer_name  IN customers.customer_name%TYPE,
        p_phone          IN customers.phone%TYPE,
        p_city           IN customers.city%TYPE,
        p_total_purchase IN customers.total_purchase%TYPE
    );

    PROCEDURE update_customer_loyalty
    (
        p_customer_id IN customers.customer_id%TYPE
    );

    PROCEDURE show_customer
    (
        p_customer_id IN customers.customer_id%TYPE
    );

    FUNCTION get_loyalty_level
    (
        p_total_purchase IN NUMBER
    )
    RETURN VARCHAR2;

    FUNCTION get_customer_city
    (
        p_customer_id IN customers.customer_id%TYPE
    )
    RETURN VARCHAR2;
END customer_util_pkg;
/
```

```
CREATE OR REPLACE PACKAGE BODY customer_util_pkg
AS
    g_package_name VARCHAR2(100) := 'Customer Utility Package';

    FUNCTION get_loyalty_level
    (
        p_total_purchase IN NUMBER
    )
    RETURN VARCHAR2
    AS
    BEGIN
        IF p_total_purchase >= 5000000 THEN
            RETURN 'PLATINUM';
        ELSIF p_total_purchase >= 2000000 THEN
            RETURN 'GOLD';
        ELSIF p_total_purchase >= 1000000 THEN
            RETURN 'SILVER';
        ELSE
            RETURN 'BASIC';
        END IF;
    END get_loyalty_level;

    FUNCTION get_customer_city
    (
        p_customer_id IN customers.customer_id%TYPE
    )
    RETURN VARCHAR2
    AS
        v_city customers.city%TYPE;
    BEGIN
        SELECT city
        INTO v_city
        FROM customers
        WHERE customer_id = p_customer_id;

        RETURN v_city;

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN 'NOT FOUND';
    END get_customer_city;

    PROCEDURE add_customer
    (
        p_customer_id    IN customers.customer_id%TYPE,
        p_customer_name  IN customers.customer_name%TYPE,
        p_phone          IN customers.phone%TYPE,
        p_city           IN customers.city%TYPE,
        p_total_purchase IN customers.total_purchase%TYPE
    )
    AS
        v_loyalty_level customers.loyalty_level%TYPE;
    BEGIN
```

```
    v_loyalty_level := get_loyalty_level(p_total_purchase);

    INSERT INTO customers
    (
        customer_id,
        customer_name,
        phone,
        city,
        total_purchase,
        loyalty_level,
        status
    )
    VALUES
    (
        p_customer_id,
        p_customer_name,
        p_phone,
        p_city,
        p_total_purchase,
        v_loyalty_level,
        g_default_status
    );

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Customer added successfully.');
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Customer ID already exists.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Customer Package Error: ' || SQLERRM);
END add_customer;

PROCEDURE update_customer_loyalty
(
    p_customer_id IN customers.customer_id%TYPE
)
AS
    v_total_purchase customers.total_purchase%TYPE;
    v_loyalty_level  customers.loyalty_level%TYPE;
BEGIN
    SELECT total_purchase
    INTO v_total_purchase
    FROM customers
    WHERE customer_id = p_customer_id;

    v_loyalty_level := get_loyalty_level(v_total_purchase);

    UPDATE customers
    SET loyalty_level = v_loyalty_level
    WHERE customer_id = p_customer_id;

    COMMIT;
```

```
        DBMS_OUTPUT.PUT_LINE('Customer loyalty updated.');
        DBMS_OUTPUT.PUT_LINE('Loyalty Level: ' || v_loyalty_level);

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Customer not found.');
    END update_customer_loyalty;

    PROCEDURE show_customer
    (
        p_customer_id IN customers.customer_id%TYPE
    )
    AS
        v_customer customers%ROWTYPE;
    BEGIN
        SELECT *
        INTO v_customer
        FROM customers
        WHERE customer_id = p_customer_id;

        DBMS_OUTPUT.PUT_LINE('===== Customer Information =====');
        DBMS_OUTPUT.PUT_LINE('Package: ' || g_package_name);
        DBMS_OUTPUT.PUT_LINE('Customer ID: ' || v_customer.customer_id);
        DBMS_OUTPUT.PUT_LINE('Customer Name: ' ||
v_customer.customer_name);
        DBMS_OUTPUT.PUT_LINE('Phone: ' || v_customer.phone);
        DBMS_OUTPUT.PUT_LINE('City: ' || v_customer.city);
        DBMS_OUTPUT.PUT_LINE('Total Purchase: ' ||
v_customer.total_purchase);
        DBMS_OUTPUT.PUT_LINE('Loyalty Level: ' ||
v_customer.loyalty_level);
        DBMS_OUTPUT.PUT_LINE('Status: ' || v_customer.status);

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Customer not found.');
    END show_customer;

BEGIN
    DBMS_OUTPUT.PUT_LINE('Customer Utility Package initialized.');
END customer_util_pkg;
/

/* Calling Customer Utility Package */
BEGIN
    customer_util_pkg.update_customer_loyalty(1);
    customer_util_pkg.update_customer_loyalty(2);
    customer_util_pkg.update_customer_loyalty(3);

    customer_util_pkg.add_customer
    (
        4,
        'Future Digital Co., Ltd',
        '094444444',
```

```
        'Naypyidaw',
        3000000
    );

    customer_util_pkg.show_customer(4);

    DBMS_OUTPUT.PUT_LINE(
        'Customer City: ' || customer_util_pkg.get_customer_city(4)
    );

    DBMS_OUTPUT.PUT_LINE(
        'Loyalty for 6,000,000: ' ||
        customer_util_pkg.get_loyalty_level(6000000)
    );
END;
/
```

```
 Customer Utility Package initialized.
Customer loyalty updated.
Loyalty Level: PLATINUM
Customer loyalty updated.
Loyalty Level: GOLD
Customer loyalty updated.
Loyalty Level: BASIC
Customer added successfully.
===== Customer Information =====
Package: Customer Utility Package
Customer ID: 4
Customer Name: Future Digital Co., Ltd
Phone: 0944444444
City: Naypyidaw
Total Purchase: 3000000
Loyalty Level: GOLD
Status: ACTIVE
Customer City: Naypyidaw
Loyalty for 6,000,000: PLATINUM

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   FINAL CHECKING
   ============================================================ */
```

```
SQL> SELECT * FROM employees ORDER BY employee_id;

EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------------------
DEPARTMENT                                          SALARY   BONUS
--------------------------------------------------- ---------- ---------
STATUS
--------------------
         1
Aung Aung
IT                                                  900000   90000
ACTIVE


EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------------------
DEPARTMENT                                          SALARY   BONUS
--------------------------------------------------- ---------- ---------
STATUS
--------------------
         2
Mg Mg
HR                                                  600000       0
ACTIVE
```

```
SQL> SELECT * FROM customers ORDER BY customer_id;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------
PHONE
------------------------------
CITY                                               TOTAL_PURCHASE
-------------------------------------------------- --------------
LOYALTY_LEVEL        STATUS
-------------------- --------------------
           1
ABC Trading
091111111

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------
PHONE
------------------------------
CITY                                               TOTAL_PURCHASE
-------------------------------------------------- --------------
LOYALTY_LEVEL        STATUS
-------------------- --------------------
Yangon                                                  5000000
PLATINUM             ACTIVE
```